

Neural Models: Learning, Depth, Activations, and Output Layers

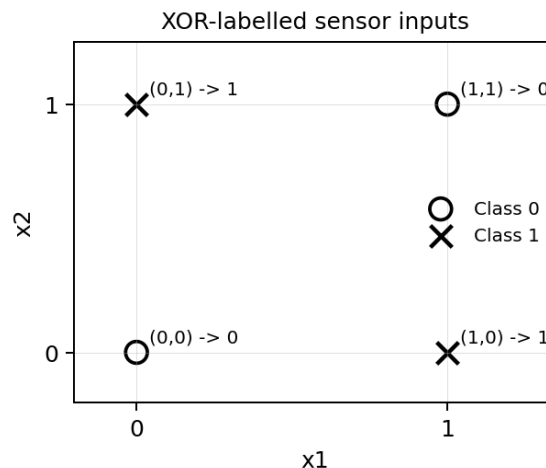
Hands-on Laboratory Report

Name: Sidh Virmani ID: 2024A7PS0520G

Repository: 2024A7PS0520G_AI_HandsOn / 1_Neural_Models

1. Task 1 - Problem specification and linear separability

The input space is $X = \{0,1\}^2$. The output space is $Y = \{0,1\}$, where 1 means the two redundant sensors disagree. The labelled examples are $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, and $(1,1) \rightarrow 0$.



Why a straight boundary fails. The two positive examples lie on opposite corners, while the two negative examples occupy the other two corners. No single straight line can place both positive points on one side and both negative points on the other.

Prediction for affine + sigmoid. A single affine transformation followed by a sigmoid cannot represent XOR exactly. Training may reduce the loss to a compromise, but it cannot correctly separate all four points because the decision boundary remains linear.

Think About It. XOR tests the claim that adding parameters or affine-only depth is not enough; a model needs a representation capable of nonlinearity to express this decision rule.

2. Task 2 - Model design and validation criteria

Baseline: 2 inputs $\rightarrow$ 2 hidden units $\rightarrow$ 1 output logit. The hidden activation is tanh. The binary output is trained with BCEWithLogitsLoss, which combines a sigmoid-compatible binary objective with numerically stable logit handling. Optimisation uses full-batch Adam.

Check	Evidence of successful learning
Loss	Final loss is substantially below the initial loss.
Predictions	All four XOR examples are classified correctly at threshold 0.5.
Backpropagation	The first-layer gradient after backward() is finite and nonzero.
Reproducibility	A fixed seed gives a repeatable run for verification.

Why the hidden nonlinearity is necessary. Without it, composing the two affine layers still gives one affine map, so the network would still have a linear decision boundary. **Why sigmoid/BCE is appropriate.** The target is one binary yes/no variable, so the output needs a binary probability interpretation and a binary classification loss.

Think About It. Hidden units have no direct target labels. Their useful features emerge because backpropagation sends the output loss gradient backward through the network, changing hidden-layer parameters in directions that reduce the final prediction error.

3. Task 3 - LLM prompts and implementation verification

PROMPT 1 - Binary XOR implementation

Generate minimal PyTorch code for the following exact experiment. Use the four XOR examples (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. Use a 2-input -> 2-hidden-unit -> 1-output network, tanh as the hidden activation, and one output logit trained with BCEWithLogitsLoss. Use PyTorch's random initialisation, set a fixed random seed, train full-batch on CPU for a few thousand lightweight steps with a gradient-based optimiser, and do not change the architecture or task. Report the initial and final loss, all four sigmoid probabilities, thresholded labels, and the first-layer weight gradient immediately after backward(). Keep the code concise and commented.

PROMPT 2 - Required diagnostics

Using the same XOR architecture and dataset, add the lab diagnostics without changing the scientific task. First, run a symmetry experiment in which all trainable parameters are initialised to zero and print the two rows of the first hidden-layer weight matrix at several training steps so we can check whether they remain identical. Second, repeat the randomly initialised experiment with sigmoid, tanh, and ReLU hidden activations, changing only the activation, and report for each run the final loss, whether all four labels are correct, and the Euclidean norm of the first-layer gradient after the first backward pass.

PROMPT 3 - Three-class extension

Modify only the output/loss portion of the small neural model for a three-class version of the same sensor task: class 0 for (0,0), class 1 for (0,1) and (1,0), and class 2 for (1,1). Keep the 2-unit tanh hidden layer, replace the single output with three logits, and use CrossEntropyLoss. Train on the four examples, then report the output weight-matrix shape, all four softmax probability vectors and predicted classes. For one example, print the sum of the softmax probabilities to verify that it is approximately 1.

Corrections/refinements made during verification:

1. Probability reporting was separated from training: the model returns logits, and sigmoid is applied only when binary probabilities are printed. 2. The first-layer gradient is captured immediately after backward() and before optimizer.step(), so the recorded tensor is the gradient produced by that pass.

Static checks before execution: dataset values, 2-2-1 dimensions, tanh hidden activation, logit output, BCEWithLogitsLoss, full-batch loop, backward(), and optimizer.step() can all be verified by reading Code.py. **Runtime checks:** actual loss reduction, predictions, gradient magnitudes, zero-initialisation behaviour, and activation outcomes require execution and measurement.

The final executable implementation is **Code.py**, submitted alongside this report. It contains the binary experiment, gradient check, symmetry experiment, activation comparison, and three-class extension.

4. Task 4 - Execution, gradients, symmetry, and activations

4.1 Basic learning and backpropagation check

Measurement	Observed value
Initial binary loss	0.693229
Final binary loss	0.000090
Final probabilities	(0,0): 0.000070; (0,1): 0.999901; (1,0): 0.999876; (1,1): 0.000068
Thresholded labels	0, 1, 1, 0 - all 4 correct
Early grad W1 _2	0.005077

The first-layer gradient after the first backward pass was $\begin{bmatrix} -0.0006 & 0.0005 \\ 0.0021 & 0.0046 \end{bmatrix}$ (rounded).

parameter.grad stores dL/dW for the parameter after reverse-mode automatic differentiation. Because BCEWithLogitsLoss uses the mean over the four examples, this gradient is the average of the four example-wise gradient contributions.

The useful learning signal is demonstrated by more than nonzero-ness: the loss fell from about 0.693 to 0.00009 and the same updates produced the correct label for every example.

4.2 Zero-initialisation symmetry experiment

Step	Hidden row 1	Hidden row 2	Identical?
0	[0.0, 0.0]	[0.0, 0.0]	Yes
1	[0.0, 0.0]	[0.0, 0.0]	Yes
10	[0.0, 0.0]	[0.0, 0.0]	Yes
100	[0.0, 0.0]	[0.0, 0.0]	Yes
500	[0.0, 0.0]	[0.0, 0.0]	Yes

Final loss remained 0.693147 and all probabilities remained 0.5. Identically initialised hidden units compute the same values and receive symmetric learning signals, so symmetry is not broken. In this fully zeroed case the network is stuck at an especially symmetric stationary configuration.

4.3 Activation experiment

Hidden activation	Final loss	4/4 correct?	Early $\ grad\ W1\ _2$
Sigmoid	0.000259	Yes	0.005453
Tanh	0.000090	Yes	0.005077
ReLU	0.000033	Yes	0.004868

All three activations learned this tiny run, but their measured losses and early gradient norms differed. These numbers describe this experiment only; they do not establish a universally best activation. Scientifically, activation derivatives alter gradient propagation. Engineering-wise, the observed values above are empirical outcomes for this seed, optimiser, and learning rate.

Think About It. To distinguish sigmoid saturation from a dead ReLU, inspect hidden pre-activation z and activation h . A saturated sigmoid has large $|z|$ with h close to 0 or 1 and a tiny derivative. A dead ReLU has $z < 0$, $h = 0$, and derivative 0.

5. Task 5 - Three-class extension

The hidden representation remains 2 units with tanh. Only the output task is changed: the final layer has **3 logits** and training uses multiclass cross-entropy. The final weight matrix therefore has shape **(3, 2)**. Each example produces three logits.

Softmax converts logits z_k into $\exp(z_k)$ divided by the sum of $\exp(z_j)$, so the components are nonnegative and sum to 1. For softmax followed by cross-entropy, differentiating with respect to the logits simplifies to $\mathbf{p} - \mathbf{y}$, where $\mathbf{y}$ is the one-hot target vector.

Input	Class probabilities [P0, P1, P2]	Pred.	Target
(0,0)	[0.999973, 0.000027, 0.000000]	0	0
(0,1)	[0.000009, 0.999985, 0.000006]	1	1
(1,0)	[0.000008, 0.999985, 0.000007]	1	1
(1,1)	[0.000000, 0.000024, 0.999976]	2	2

Initial multiclass loss = 1.040851; final loss = 0.000020. The softmax probability sum for input (0,1) was numerically 1.00000000.

Think About It. Large-vocabulary next-token prediction keeps the same core mathematics: one logit per class/token, softmax normalisation, cross-entropy, and the p-y logit gradient. What changes dramatically is the surrounding model: vocabulary size, embedding/output matrices, sequence context, attention, depth, compute, and training data.

6. Reflection questions

1. What did XOR demonstrate about depth and nonlinearity?

Affine-only depth still collapses to one affine map, so depth alone does not solve XOR. A nonlinear hidden activation changes the representational class and allows the 2-2-1 network to learn XOR.

2. What showed that backpropagation supplied a useful learning signal?

The first-layer gradient was nonzero, but more importantly the loss dropped from 0.693229 to 0.000090 and all four predictions became correct. The gradient therefore produced useful parameter updates, not merely numerical activity.

3. Why did identical/zero initialisation prevent distinct hidden features?

Identical hidden units start with the same computation and receive symmetric gradients, so they remain identical. They cannot spontaneously specialise into different features; in the fully zeroed run the model remained stuck at 0.5 outputs.

4. How did hidden activation affect the observed gradient?

The early first-layer gradient norms were 0.005453 (sigmoid), 0.005077 (tanh), and 0.004868 (ReLU). Scientifically, different activation derivatives alter gradient flow; engineering-wise, these specific values are measurements for this one controlled run.

5. Why must the output layer and loss be selected together?

They must match the prediction semantics. Binary XOR uses one logit with binary cross-entropy; the three-class task uses three logits with multiclass cross-entropy. Mismatching them would encode the wrong statistical task or require incorrect target shapes.

6. Where did the LLM help, and where was human verification essential?

The LLM accelerated boilerplate PyTorch implementation and the diagnostic scaffolding. Human verification was essential for checking the architecture, measuring actual gradients and predictions, and confirming that the zero-initialisation and multiclass experiments behaved as required.

7. Which tests would be kept at larger scale?

Keep automated loss/metric checks, held-out predictions, gradient statistics, shape tests, reproducibility checks, and small targeted unit tests. Exhaustive checks over all inputs and full finite-difference gradients become too expensive as model and dataset size grow.

7. Conclusion

The experiment connects representation and learning: XOR requires a nonlinear hidden representation, backpropagation supplies the parameter gradients, and engineering checks are needed to verify that learning actually occurred. The same small model also shows why symmetry-breaking initialisation matters and why the output/loss pair must change when the task changes from binary to multiclass classification.